



UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

FACULTAD DE CIENCIAS DE LA COMPUTACIÓN

CARRERA INGENIERÍA SOFTWARE

ASIGNATURA:

APLICACIONES WEB

NOMBRE:

IRVIN MARCELO CAJAS IBARRA

FECHA:

JUEVES, 04 DE JUNIO DEL 2026

(Semana V)

1. Introducción

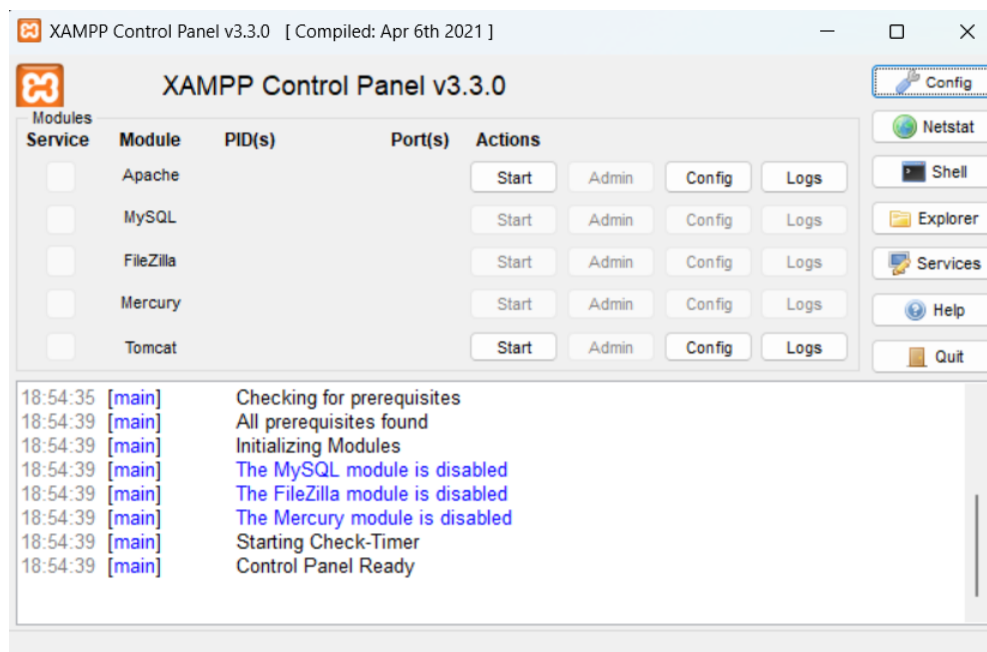
El procesamiento seguro de formularios constituye una tarea fundamental en el desarrollo de aplicaciones web. Una validación deficiente puede provocar errores de funcionamiento o permitir vulnerabilidades que comprometan la información almacenada. Diversas investigaciones coinciden en que los problemas relacionados con la validación de entradas continúan siendo una de las causas más frecuentes de fallos de seguridad en aplicaciones web [1], [2].

En esta práctica se implementó un formulario HTML común para ser procesado mediante PHP 8.2 y PERL/CGI. El objetivo de la práctica no fue solamente comprobar que ambos lenguajes procesan formularios correctamente. También se buscó analizar como manejan la validación de datos y que tan efectivos resultan frente a situaciones comunes, como el ingreso de información incorrecta o intentos básicos de XSS [3], [4].

2. Instalación del entorno

Se instaló XAMPP 8.2.12 y se habilitaron los servicios necesarios para ejecutar aplicaciones PHP y PERL/CGI. Posteriormente se verificó el funcionamiento correcto del entorno mediante la revisión de versiones y la ejecución de pruebas simples en ambos lenguajes.

2.1. Panel de control de XAMPP con Apache activo



2.2. Salida de PHP en consola

```
C:\Users\irvin>cd C:\xampp\php  
  
C:\xampp\php>php.exe -v  
PHP 8.2.12 (cli) (built: Oct 24 2023 21:15:15) (ZTS Visual C++ 2019 x64)  
Copyright (c) The PHP Group  
Zend Engine v4.2.12, Copyright (c) Zend Technologies
```

2.3. Salida de PERL en consola

```
C:\xampp\php>cd C:\xampp\perl\bin  
  
C:\xampp\perl\bin>perl.exe -v  
  
This is perl 5, version 32, subversion 1 (v5.32.1) built for MSWin32-x64-multi-thread  
Copyright 1987-2021, Larry Wall  
  
Perl may be copied only under the terms of either the Artistic License or the  
GNU General Public License, which may be found in the Perl 5 source kit.  
  
Complete documentation for Perl, including FAQ lists, should be found on  
this system using "man perl" or "perldoc perl". If you have access to the  
Internet, point your browser at http://www.perl.org/, the Perl Home Page.
```

3. Formulario HTML

Se desarrollo un formulario HTML compuesto por los campos nombre, correo electrónico, edad y mensaje. El mismo formulario fue utilizado para ambos lenguajes con el fin de garantizar que las pruebas se realizaran bajo las mismas condiciones de entrada.



The screenshot shows a web browser window with the address bar displaying 'localhost/xampp/practica5/formulario.html'. The page title is 'Formulario de Registro'. The form contains four input fields: 'Nombre completo:', 'Correo electrónico:', 'Edad:', and 'Mensaje:'. At the bottom, there are two buttons: 'Enviar con PHP' and 'Enviar con PERL/CGI'.

← → ↻ 🏠 ⓘ localhost/xampp/practica5/formulario.html

Formulario de Registro

Nombre completo:

Correo electrónico:

Edad:

Mensaje:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Práctica 5 - PHP y PERL/CGI</title>
</head>
<body>

  <h1>Formulario de Registro</h1>

  <form method="POST">

    <p>
      <label>Nombre completo:</label><br>
      <input type="text" name="nombre">
    </p>

    <p>
      <label>Correo electrónico:</label><br>
      <input type="email" name="email">
    </p>

    <p>
      <label>Edad:</label><br>
      <input type="number" name="edad" min="1" max="120">
    </p>

    <p>
      <label>Mensaje:</label><br>
      <textarea name="mensaje" rows="4" cols="40"></textarea>
    </p>

    <button type="submit" formaction="procesar.php">
      Enviar con PHP
    </button>

    <button type="submit" formaction="/cgi-bin/procesar.pl">
      Enviar con PERL/CGI
    </button>

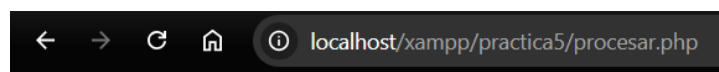
  </form>

</body>
</html>
```

4. Implementación en PHP

La versión desarrollada en PHP recibe los datos enviados mediante el método POST y realiza las validaciones correspondientes antes de procesarlos. Para la comprobación de datos se emplearon funciones integradas del lenguaje, mientras que la salida fue saneada para impedir la ejecución de contenido potencialmente peligroso [1], [2].

Durante las pruebas se revisaron situaciones frecuentes en cualquier formulario web. Por ejemplo, se comprobaron campos vacíos, correos mal escritos y edades fuera de los límites establecidos para verificar que el sistema respondiera de manera adecuada.[5].



Resultado PHP

Datos válidos

Nombre: Irvin Cajas

Email: icajasi@uteq.edu.ec

Edad: 20

Mensaje: Estudiante de Software

[Volver](#)

```

<?php
$errores = [];

$nombre = filter_input(INPUT_POST, 'nombre');
$email = filter_input(INPUT_POST, 'email', FILTER_VALIDATE_EMAIL);

$edad = filter_input(
    INPUT_POST,
    'edad',
    FILTER_VALIDATE_INT,
    [
        "options" => [
            "min_range" => 1,
            "max_range" => 120
        ]
    ]
);

$mensaje = filter_input(INPUT_POST, 'mensaje');

$nombre = htmlspecialchars(trim($nombre ?? ""));
$mensaje = htmlspecialchars(trim($mensaje ?? ""));

if ($nombre == "")
{
    $errores[] = "Nombre vacío";
}

if ($email === false || $email === null)
{
    $errores[] = "Correo inválido";
}

if ($edad === false || $edad === null)
{
    $errores[] = "Edad inválida";
}

if ($mensaje == "")
{
    $errores[] = "Mensaje vacío";
}
?>

<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <title>Resultado PHP</title>
</head>
<body>

<h1>Resultado PHP</h1>

<?php
if (count($errores) > 0)
{
    echo "<h3>Errores encontrados:</h3>";

    foreach ($errores as $e)
    {

```

```

        echo "<p>$e</p>";
    }
}
else
{
    echo "<h3>Datos válidos</h3>";

    echo "<p>Nombre: $nombre</p>";
    echo "<p>Email: $email</p>";
    echo "<p>Edad: $edad</p>";
    echo "<p>Mensaje: $mensaje</p>";
}
?>

<br>
<a href="formulario.html">Volver</a>

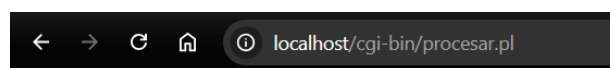
</body>
</html>

```

5. Implementación en PERL/CGI

En PERL se utilizó CGI.pm para recibir los datos enviados desde el formulario. Aunque el enfoque de programación fue diferente al de PHP, se aplicaron las mismas reglas de validación con el fin de comparar ambos entornos bajo condiciones similares.

También se aplicaron mecanismos de saneamiento para evitar que etiquetas HTML o scripts fueran interpretados por el navegador. Este tipo de medidas sigue siendo una defensa esencial frente a ataques XSS en aplicaciones web [3], [6].



Resultado PERL

Datos validos

Nombre: Irvin Marcelo

Email: icajasi@uteq.edu.ec

Edad: 20

Mensaje: Estudiante de la UTEQ

[Volver](#)

```

#!/C:/xampp/perl/bin/perl.exe

use strict;
use warnings;
use CGI;

my $cgi = CGI->new;

print $cgi->header('text/html; charset=UTF-8');

my $nombre = $cgi->param('nombre') || '';
my $email = $cgi->param('email') || '';
my $edad = $cgi->param('edad') || '';
my $mensaje = $cgi->param('mensaje') || '';

my @errores;

if ($nombre eq '')
{
    push @errores, "Nombre vacio";
}

if ($email !~ /^[^\\s\\@]+\\@[^\\s\\@]+\\.([^\\s\\@]+)$/ )
{
    push @errores, "Correo invalido";
}

if ($edad !~ /^[^\\d]+$/ || $edad < 1 || $edad > 120)
{
    push @errores, "Edad invalida";
}

if ($mensaje eq '')
{
    push @errores, "Mensaje vacio";
}

$nombre = $cgi->escapeHTML($nombre);
$email = $cgi->escapeHTML($email);
$edad = $cgi->escapeHTML($edad);
$mensaje = $cgi->escapeHTML($mensaje);

print "<html>";
print "<head><title>Resultado PERL</title></head>";
print "<body>";

print "<h1>Resultado PERL</h1>";

if (@errores)
{
    print "<h3>Errores encontrados:</h3>";

    foreach my $e (@errores)
    {
        print "<p>$e</p>";
    }
}
else
{
    print "<h3>Datos validos</h3>";

    print "<p>Nombre: $nombre</p>";
}

```



```
print "<p>Email: $email</p>";
print "<p>Edad: $edad</p>";
print "<p>Mensaje: $mensaje</p>";
}

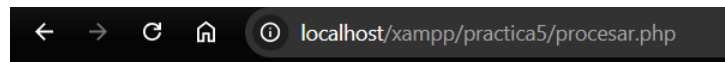
print '<br><a href="/xampp/practica5/formulario.html">Volver</a>';

print "</body>";
print "</html>";
```

6. Casos de prueba

Se ejecutaron los diez casos definidos para la practica con el objetivo de comprobar el comportamiento de ambos programas frente a entradas validas e invalidas. Los resultados obtenidos permitieron verificar el correcto funcionamiento de las validaciones implementadas.

6.1. Caso 1 - Campos vacíos



Resultado PHP

Errores encontrados:

Nombre vacío

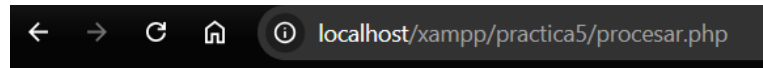
Correo inválido

Edad inválida

Mensaje vacío

[Volver](#)

6.2. Caso 2 - Correo sin dominio



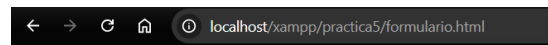
Resultado PHP

Errores encontrados:

Correo inválido

[Volver](#)

6.3. Caso 3 - Correo sin arroba



Formulario de Registro

Nombre completo:

irvin

Correo electrónico:

juangmail.com



Incluye un signo "@" en la dirección de correo electrónico. La dirección "juangmail.com" no incluye el signo "@".

Enviar con PHP

Enviar con PERL/CGI

6.4. Caso 4 - Edad negativa



A screenshot of a web browser displaying a registration form titled "Formulario de Registro". The browser's address bar shows "localhost/xampp/practica5/formulario.html". The form contains three input fields: "Nombre completo:" with the value "irvin", "Correo electrónico:" with the value "Irvincajas@gmail.com", and "Edad:" with a spinner control set to "-5". A red error message box is visible below the age field, stating "El valor debe ser superior o igual a 1". At the bottom of the form, there are two buttons: "Enviar con PHP" and "Enviar con PERL/CGI".

6.5. Caso 5 - Edad fuera de rango



A screenshot of a web browser displaying a registration form titled "Formulario de Registro". The browser's address bar shows "localhost/xampp/practica5/formulario.html". The form contains three input fields: "Nombre completo:" with the value "irvin", "Correo electrónico:" with the value "Irvincajas@gmail.com", and "Edad:" with a spinner control set to "-5". A red error message box is visible below the age field, stating "El valor debe ser superior o igual a 1". At the bottom of the form, there are two buttons: "Enviar con PHP" and "Enviar con PERL/CGI".

6.6. Caso 6 - Gmail con espacio



Formulario de Registro

Nombre completo:
irvin

Correo electrónico:
Irvincajas@gmail.com

Edad:
200

El valor debe ser inferior o igual a 120

Enviar con PHP Enviar con PERL/CGI

6.7. Caso 7 - Intento XSS en nombre



Resultado PHP

Datos válidos

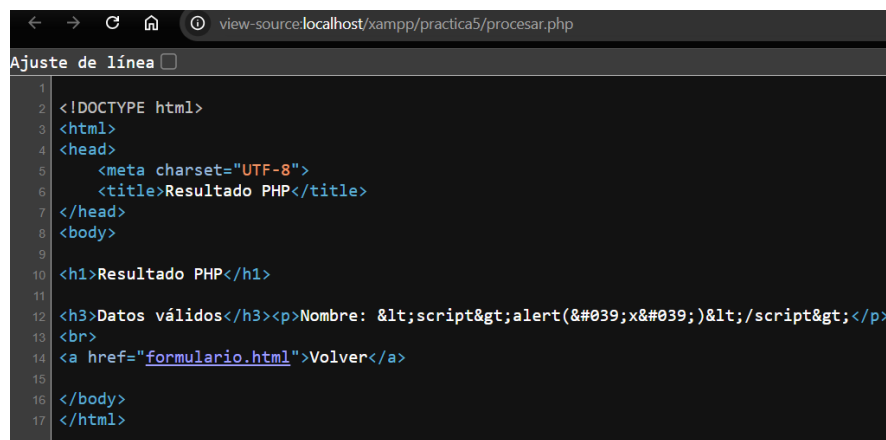
Nombre: <script>alert('x')</script>

Email: irvin@gmail.com

Edad: 20

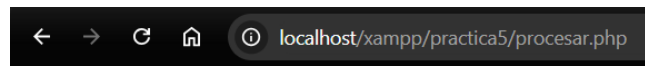
Mensaje: Software

[Volver](#)



```
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <meta charset="UTF-8">
5   <title>Resultado PHP</title>
6 </head>
7 <body>
8   <h1>Resultado PHP</h1>
9   <h3>Datos válidos</h3><p>Nombre: &lt;script&gt;alert(&#039;x&#039;)&lt;/script&gt;</p>
10  <br>
11  <a href="formulario.html">Volver</a>
12 </body>
13 </html>
```

6.8. Caso 8 - Intento XSS en mensaje



Resultado PHP

Datos válidos

Nombre: <script>alert('x')</script>

Email: irvin@gmail.com

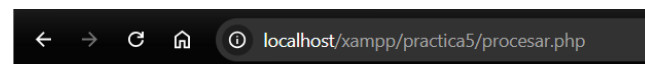
Edad: 20

Mensaje: Software

[Volver](#)

```
view-source:localhost/xampp/practica5/procesar.php
Ajuste de líneas
1 <!DOCTYPE html>
2 <html>
3 <head>
4 <meta charset="UTF-8">
5 <title>Resultado PHP</title>
6 </head>
7 <body>
8
9 <h1>Resultado PHP</h1>
10
11 <h3>Datos válidos</h3><p>Nombre: irvin cajas</p><p>Email: irvincajas@gmail.com</p><p>Edad: 20</p><p>Mensaje: &lt;script&gt;alert(&#039;x&#039;)&lt;/script&gt;</p>
12 <br>
13 <a href="formulario.html">Volver</a>
14
15 </body>
16 </html>
```

6.9. Caso 9 - Datos válidos



Resultado PHP

Datos válidos

Nombre: Irvin Cajas Ibarra

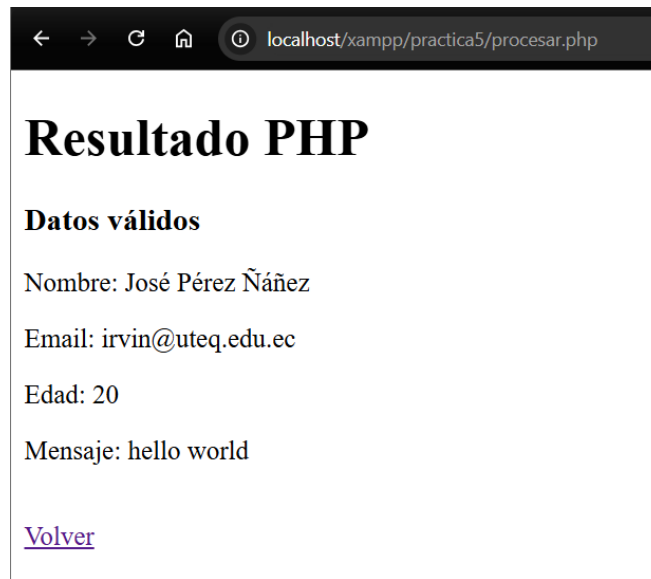
Email: icajasi@uteq.edu.ec

Edad: 20

Mensaje: hola mundo

[Volver](#)

6.10. Caso 10 - Nombre con caracteres especiales



7. Tabla comparativa PHP vs. PERL/CGI

Tabla 1: Comparación entre PHP 8.2 y PERL/CGI.

Criterio	PHP 8.2	PERL/CGI	Observación personal
Instalación	Rápida y sencilla	Requiere configuración adicional	PHP estuvo listo para usarse en menos pasos.
Desarrollo	Mas directo	Mas detallado	La implementación en PHP resulto más intuitiva.
Validación de formularios	Funciones integradas	Implementación manual	PHP redujo la cantidad de código necesario.
Protección frente a XSS	Efectiva	Efectiva	Ambos lenguajes bloquearon correctamente las pruebas realizadas.
Mensajes de error	Mas claros	Mas técnicos	Fue más fácil identificar problemas en PHP.
Legibilidad del código	Alta	Media	El código PHP fue mas fácil de comprender y mantener.
Tiempo de desarrollo	Menor	Mayor	PERL requirió mas configuración y pruebas iniciales.
Uso actual	Muy extendido	Mas especializado	PHP tiene una presencia mucho mayor en desarrollo web [2].
Aplicaciones recomendadas	Sitios web, APIs y CMS	Automatización y procesamiento de texto	Cada tecnología destaca en escenarios diferentes [5].
Experiencia durante la practica	Mas cómoda	Mas exigente	Para un proyecto web similar elegiría PHP.

8. Conclusión

La práctica permitió verificar el funcionamiento de PHP 8.2 y PERL/CGI en el procesamiento seguro de formularios web. En ambos casos se implementaron mecanismos para validar los datos recibidos, rechazar entradas incorrectas y evitar la ejecución de contenido potencialmente peligroso. Los resultados obtenidos demostraron que las validaciones aplicadas fueron suficientes para detectar campos vacíos, correos electrónicos inválidos, edades fuera del rango establecido e intentos básicos de XSS.

La diferencia mas significativa observada durante el desarrollo estuvo relacionada con la facilidad de implementación. PHP proporciono una experiencia mas directa gracias a sus funciones integradas para validación y saneamiento de datos, lo que permitió desarrollar la solución con menos código y menor tiempo de configuración. Por su parte, PERL ofreció resultados equivalentes, aunque requirió una mayor intervención manual durante el proceso de desarrollo.

A partir de la experiencia realizada, elegiría PHP para proyectos orientados al desarrollo web debido a su simplicidad, integración con servidores y facilidad de mantenimiento. Sin embargo, PERL sigue siendo una alternativa útil cuando se requiere automatización, procesamiento de texto o tareas especializadas. Teniendo en cuenta la comparación entre lenguajes, la práctica permitió comprobar que una validación adecuada sigue siendo uno de los elementos mas importantes para desarrollar aplicaciones web seguras. Incluso formularios sencillos pueden presentar riesgos cuando los datos ingresados por los usuarios no son tratados correctamente.

Referencias

- [1] M. A. Faisal y H. T. Elshoush, "Input Validation Vulnerabilities in Web Applications: Systematic Review, Classification, and Analysis of the Current State-of-the-Art," *IEEE Access*, vol. 11, págs. 37 927-37 959, 2023, Open Access (CC BY-NC-ND 4.0). [10.1109/ACCESS.2023.3266385](https://doi.org/10.1109/ACCESS.2023.3266385).
- [2] Z. Yin y S. U.-J. Lee, "Security Analysis of Web Open-Source Projects Based on Java and PHP," *Electronics*, vol. 12, n.º 12, pág. 2618, 2023, Open Access (CC BY 4.0). [10.3390/electronics12122618](https://doi.org/10.3390/electronics12122618).
- [3] M. Alsaffar et al., "Detection of Web Cross-Site Scripting (XSS) Attacks," *Electronics*, vol. 11, n.º 14, pág. 2212, 2022, Open Access (CC BY 4.0). [10.3390/electronics11142212](https://doi.org/10.3390/electronics11142212).

- [4] J. R. Tadhani, V. Vekariya, V. Sorathiya, S. Alshathri y W. El-Shafai, “Securing web applications against XSS and SQLi attacks using a novel deep learning approach,” *Scientific Reports*, vol. 14, pág. 1803, 2024, Open Access (CC BY 4.0). [10.1038/s41598-023-48845-4](https://doi.org/10.1038/s41598-023-48845-4).
- [5] J. Shahid, M. K. Hameed, I. T. Javed, K. N. Qureshi, M. Ali y N. Crespi, “A Comparative Study of Web Application Security Parameters: Current Trends and Future Directions,” *Applied Sciences*, vol. 12, n.º 8, pág. 4077, 2022, Open Access (CC BY 4.0). [10.3390/app12084077](https://doi.org/10.3390/app12084077).
- [6] R. Alhamyani y M. Alshammari, “Machine Learning-Driven Detection of Cross-Site Scripting Attacks,” *Information*, vol. 15, n.º 7, pág. 420, 2024, Open Access (CC BY 4.0). [10.3390/info15070420](https://doi.org/10.3390/info15070420).